SHENZHEN INTERNATIONAL FINTECH COMPETITION 2026 · INTERNATIONAL TRACK · TOPIC C

SudoRoute

Technical Document

An AI agent that pays international students' tuition — safely, cheaply and on time

Team SudoRoute · Rajshahi, Bangladesh · Kuala Lumpur, Malaysia · 20 October 2026
Live: sudoroute.onrender.com · Code: github.com/afiaislam/sudoroute · afiaislam2313@gmail.com

> **In one sentence:** SudoRoute verifies the invoice, picks the cheapest licensed route that arrives on time, prices currency risk, and moves money only after the student — and the family, when family money is used — approve with their own codes. **AI proposes, code disposes, humans execute.**

Contents — 1 Summary · 2 Problem and users · 3 User journeys · 4 Architecture · 5 Backend server · 6 Payment lifecycle · 7 AI layer · 8 Engines and algorithms · 9 Family and accounts · 10 Certificates and on-chain proof · 11 Failure handling and resilience · 12 Judge mode and isolation · 13 Data model and persistence · 14 API · 15 Security design · 16 Web client · 17 Testing and evaluation · 18 Deployment and configuration · 19 Limitations and roadmap · 20 Sources

1. Summary

Item	Value
Competition	2026 Shenzhen International FinTech Competition, International Track, Topic C (Cross-border & Student Finance Assistant); theme "AI as a Financial Participant"
What it does	Pays tuition end to end: invoice check → cheapest licensed route → currency-risk pricing → signed proposal → guardian and student approval → settlement tracking → signed certificate
Users	International students, their parents or guardians, university finance offices (as payees); judges via a one-click private sandbox
Stack	Python 3.10+ standard library only (server, crypto, engines); vanilla JavaScript (no build step); Google Gemini (optional); OpenTimestamps (optional)
Size	≈ 7,900 lines of Python in 44 files; 12 HTML pages; 230 automated tests
Key results	Security Lab 63/63 · scam shield 100% of scams flagged, 0 genuine blocked (84 invoices) · 0 scams paid vs 42/42 for a bank wire · 1.82% cheaper than a bank wire · Bitcoin anchor confirmed in block 972087
Live	sudoroute.onrender.com · code github.com/afiaislam/sudoroute

1.1 What is new

- **AI as a bounded financial participant.** Gemini plans, reads, audits and writes in four separate roles — and none of them can move money. The fences are code (no execute tool, permission tiers, signed proposals, a policy engine run twice), so safety does not depend on the model behaving.
- **The whole job, not one step.** Invoice check, legal route choice, currency-risk pricing, family approval, settlement tracking, recovery of returned or short transfers, and a certificate at the end — in one agent.

- **Cheapest legal route that arrives on time.** Routes are ranked by landed cost including the hidden FX markup and the expected cost of arriving late; unlicensed and country-disallowed channels are excluded with a reason.
- **Two-party family approval.** When family money is used, the guardian approves with a separate code before the student's code even becomes usable.
- **Proof anyone can check.** Ed25519-signed certificates, a hash-chained audit log, and a Bitcoin timestamp through OpenTimestamps (confirmed in block 972087).
- **Reviewable in 60 seconds.** One click gives each judge a private sandbox with a student and his mother and a guided tour.

1.2 Against the alternatives

Same 84 invoices and 160 priced payments for all three, computed by `scripts/benchmark_baselines.py` (`evidence/baseline_comparison.md`).

Measure	SudoRoute	Bank wire	Cheapest-fee assistant
Scam invoices that pay a fraudster (of 42)	**0**	42	42
Payments breaking a rule (of 160)	**0**	0	120
Average saving vs bank wire	**1.82%**	0%	2.48% (only via disallowed channels)
Avoidable late payments	**0.0%**	16.5%	0.2%
Retry situations that overpay (of 3)	**0**	2	2
Family approves every family-funded payment	**Yes**	No	No

The honest cost: SudoRoute sends 2 of 42 genuine invoices to the student for a check with the university before paying.

1.3 How to check it in five minutes

1. Open sudoroute.onrender.com → **Try it now — 1 click**; follow the five-step tour (section 3.4).
2. Open `/security-lab` → **Run all attacks**: 63/63 in a few seconds, against the live server.
3. Open `/accuracy` for the scam-shield and baseline results; `/verify` to check a certificate and its Bitcoin proof.
4. Or locally: `python run.py`, then `python -m unittest discover -s tests -t .` (230 tests) and `python scripts/run_demo.py` (41/41).

1.4 Where each submission item is

Required item	Where
01 Technical documentation (architecture, algorithms, security design)	This document — sections 4–8 and 15 · `docs/SudoRoute-Technical-Document.pdf`
02 Presentation deck	`docs/SudoRoute-Pitch-Deck.pdf` (18 slides + appendix, ≤ 10 minutes)
03 Demo video	4 min 10 s (limit 5 min), on the home page and in `web/static/video/`
04 Source code (README, GitHub,	github.com/afiaislam/sudoroute — README sections 6–7, `render.yaml` +

Required item	Where
deployment, test cases)	`Dockerfile`, 230 tests in `tests/`, run on every push
05 Security self-assessment (permission tiers, known risks)	`docs/SudoRoute-Security-Self-Assessment.pdf` — tiers T0–T4, 18 known risks mapped to tests
06 Execution evidence (sandbox logs + demo video)	Execution video `evidence/execution-evidence-video.mp4` (1 min 46 s: the sandbox run, its logs and live checks) and `evidence/` (index in `evidence/README.md`): `execution_summary.json`, `sandbox_operation_log.jsonl`, `demo_transcript.md`, `security_lab_report.md`, `scam_shield_benchmark.md`, `baseline_comparison.md`, `live_llm_eval.md`, `live_gemini_transcript.md`, `onchain/`, `screenshots/`

2. Problem and users

Families paying tuition abroad face scams, hidden costs and illegal channels, and no tool checks all three before money moves.

Problem	Evidence
Scale	About 6.9 million internationally mobile students in 2022 (UNESCO). 52,799 Bangladeshi students went abroad in 2023; Bangladesh Bank recorded $730.6 million sent for study abroad in the first 10 months of FY26, up from $218 million in FY20.
Scams	Fake “agents” offer 10–20% off tuition and pay with stolen cards; the payment is reversed and the student still owes the full fee (University of Technology Sydney). Fake university e-mails announce a changed bank account (Anglia Ruskin University).
Cost	Sending $200 cost 6.49% on average in Q1 2025; banks were the costliest channel at 14.55%; the UN and G20 target is 3% by 2030 (World Bank).
Illegal channels	A father’s payment for his son’s Lancaster University fee was routed through a hawala–hundi network; the university returned the money (APP, 2 Sep 2026).

User	Needs	What SudoRoute gives them
Student	Pay on time, not overpay, not be scammed	Dashboard with due date, latest safe day to pay, on-time chance and risk of waiting; AI assistant; scam check; certificates
Parent / guardian	Know where family money goes and approve it	Own account linked by e-mail or code, approval with a separate code, kill switch, family funding, a read-only AI assistant
University	Receive the right amount with the right reference	Payments only to its registry account with the student reference; reconciliation of short payments
Judge / reviewer	See the whole system quickly and safely	One click → private sandbox with a student and his mother, a guided 60-second tour, public proof pages

Goals: complete the whole task, never pay a fraudster, choose the cheapest licensed route that arrives on time, and let AI participate without ever letting it move money on its own.

3. User journeys

3.1 Student pays tuition (family money)

1. Signs up with university, programme, home country, phone and the guardian's name and e-mail; gets a sandbox wallet, a real tuition invoice and one look-alike scam e-mail.
2. Asks the assistant, e.g. "Is my invoice real? If yes, pay it the cheapest legal way". The planner verifies the invoice, compares routes and prepares a signed proposal with 10 checks.
3. The guardian receives a one-time code on their phone and approves; only then does the student's own code become usable.
4. The student confirms; the transfer is tracked; after arrival a certificate can be issued and verified publicly.

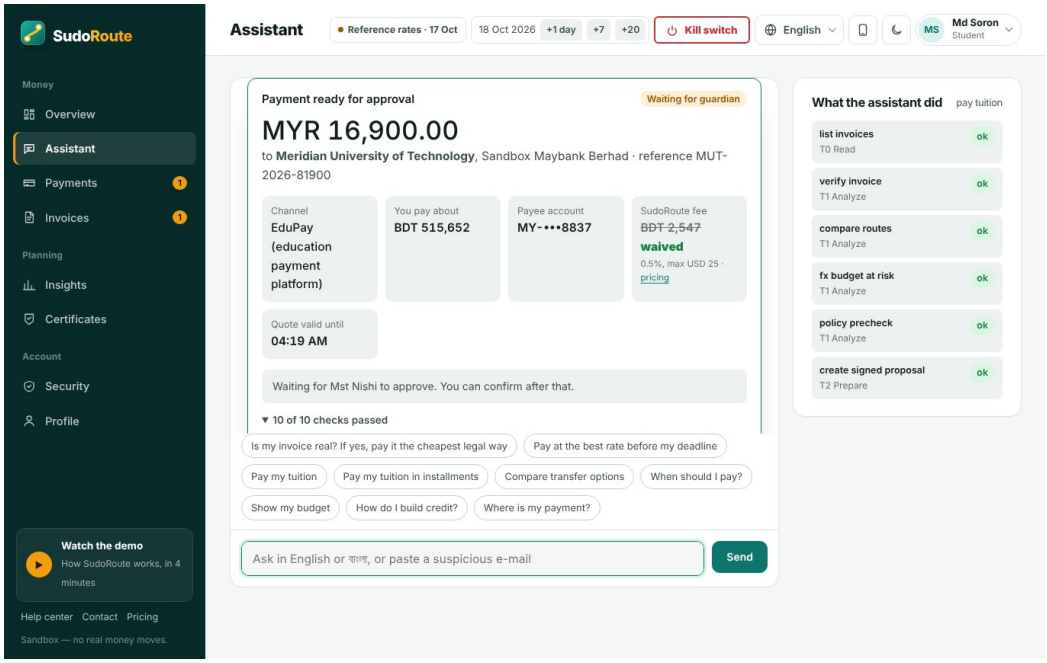


Figure — the agent has verified the invoice, compared routes and prepared a signed payment; the right panel lists every tool call with its permission tier. The payment waits for the mother.

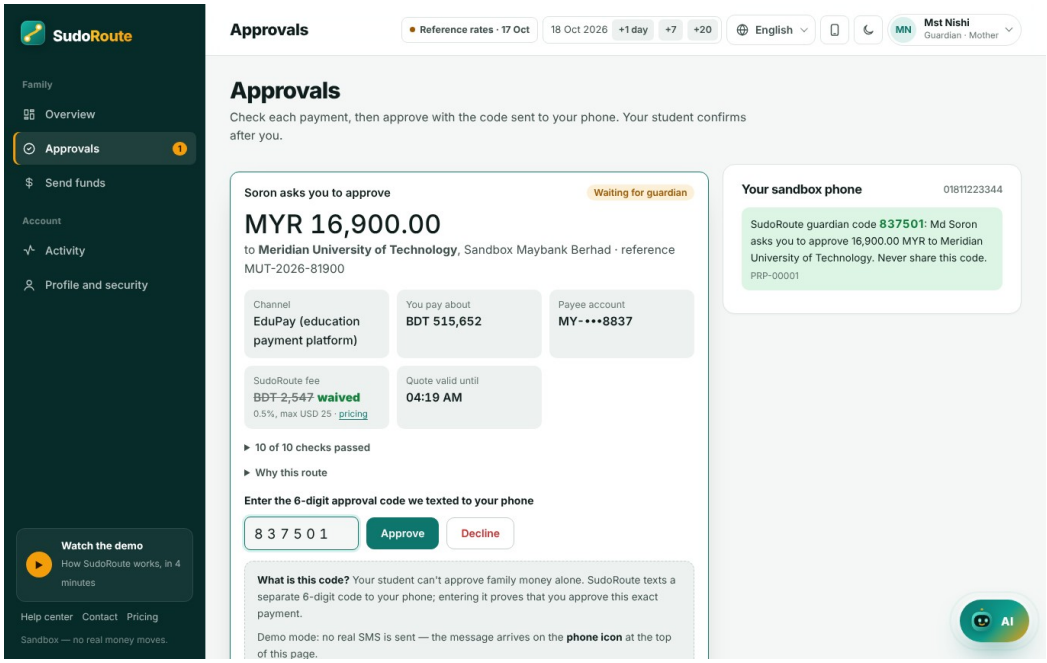


Figure — the mother's portal: she approves with the code sent to her phone; only then can the student confirm.

3.2 Smart-timing autopay

"Pay at the best rate before my deadline" → a target rate, price floor and deadline are priced and signed; one approval; the agent checks daily and pays by itself at the target or on the latest safe day, never below the floor.

3.3 Scam e-mail

A pasted e-mail is parsed, scored 0–100 with every point explained, read by the quarantined AI reader (which can only add risk), and blocked with advice to contact the university through its official website. The AI can draft a message to the real finance office; SudoRoute never sends it.

3.4 Judge

Home page → **Try it now — 1 click** → a private sandbox. A tour panel offers one button per step: ask the agent, switch to the mother and fill her code, approve, switch back and fill the student's code, confirm, move the clock +7 days, issue and download the certificate.

4. Architecture

A request flows top-down through code-enforced gates; the AI roles sit beside the path and none of them can reach execution.

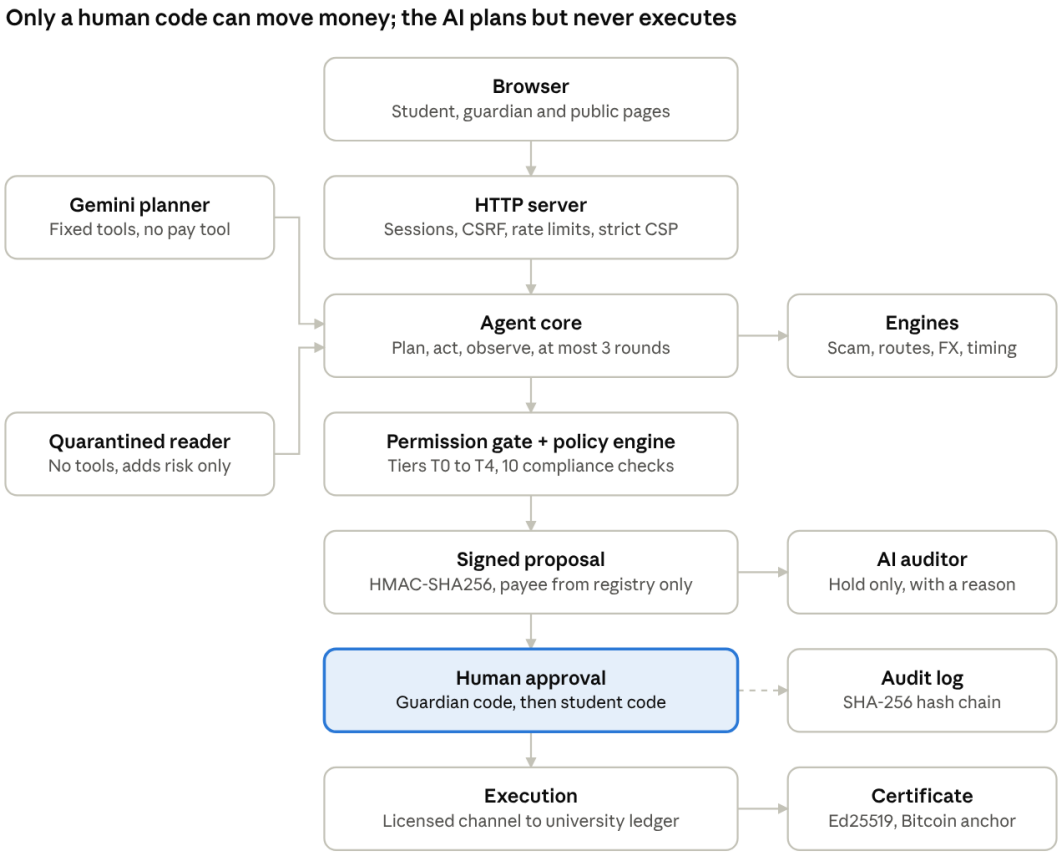


Figure 1 — SudoRoute architecture: the request path and its gates.

Package	Responsibility	Key modules
`sudoroute/agent/`	The agent: intent rules, LLM adapter, planner loop, tools, mandates, assistants	`core.py` (1,397 lines), `llm.py`, `intents.py`, `assist.py`
`sudoroute/engines/`	Deterministic engines for every number	`invoice_check.py`, `routes.py`, `fx_risk.py`, `fx_timing.py`, `obligations.py`, `budget.py`, `credit.py`
`sudoroute/security/`	Tiers, signed proposals, policy, audit, accounts, crypto, red team	`permissions.py`, `proposals.py`, `policy.py`, `audit.py`, `auth.py`, `totp.py`, `ed25519.py`, `certificates.py`, `guardrails.py`, `redteam.py`
`sudoroute/sandbox/`	Simulated bank, FX market, live FX feed, registry, reference data	`bank.py`, `fx_market.py`, `fx_live.py`, `registry.py`, `reference_data.py`
`sudoroute/chain/`	On-chain anchoring	`anchor.py` (OpenTimestamps)
`sudoroute/benchmark/`	Hold-out invoice sets and baselines	`invoice_holdout.py`, `invoice_holdout_v2.py`, `scam_shield.py`, `baselines.py`
`sudoroute/server.py`	HTTP server: routing, validation, sessions, CSRF, rate limits, headers, judge worlds	`ThreadingHTTPServer`, `App`, `World`, `WorldView`
`web/`	12 HTML pages and their scripts and styles	`index.html`, `student.html`, `guardian.html`, `lab.html`, `verify.html`, `static/*.js`, `static/site.css`

5. Backend server

SudoRoute has a real backend: one Python process (`sudoroute/server.py`, 1,075 lines) built only on the standard library — no web framework, no database server, no third-party packages. It serves the pages, owns all state and is the only place money logic runs; the browser only displays results.

Part	How it works
HTTP server	`ThreadingHTTPServer` (one thread per request) with a single `Handler`; `serve()` loads the live FX snapshot, builds the `App`, prints whether Gemini is on, and listens on `PORT` (default 8000).
App state	`App` holds the sandbox bank, FX market, registry, agent, proposal store, user store, sessions, audit log, certificate issuer and anchor store. Judge accounts get their own `World` (bank, FX, clock, agent) seen through a `WorldView`; the world is re-selected on every request so a kept-alive connection never carries another user's world.
Routing	`do_GET` serves pages, static files, video (with HTTP Range for seeking) and `/api` reads; `do_POST` checks the rate limit and origin first, then dispatches to `_login`, `_register`, `_student_post`, `_guardian_post` and public handlers by role. 62 distinct `/api` paths (section 14).
Request validation	JSON only, a JSON object only, maximum body 64 KB (320 KB only for a resized profile photo); field types, lengths, amounts and IDs checked before any engine is called; bad

Part	How it works
	input returns a short JSON error (400/403/404/429), never internal details.
Sessions and CSRF	256-bit random token in an `HttpOnly; SameSite=Strict` cookie (`Secure` behind HTTPS); 30-minute idle and 12-hour absolute expiry; every POST needs the `X-CSRF-Token` header and a matching `Origin`.
Abuse limits	120 requests per 10 seconds per IP; public AI 40/hour; contact and newsletter forms 10/hour; judge sandboxes 8/hour; Security Lab one run per 15 s; login lockouts (section 9).
Response headers	Strict CSP (`script-src 'self'`), `frame-ancestors 'none'`, `X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy: same-origin`, `Permissions-Policy` (no camera, microphone, location or payment), `Cross-Origin-Opener-Policy`, HSTS when secure; `Cache-Control: no-store` on every API and money screen.
Static delivery	All static files share one content hash (`ASSET_V`), so they are cached for a year (`immutable`) and still never go stale; ETag / 304; gzip for text, JSON and SVG over 1 KB.
Persistence	JSON files in `data/`, written atomically (temporary file + `os.replace`) under thread locks; the audit log is append-only JSON Lines with a hash chain; the issuer key is stored with 0600 permissions.
Background work	Smart-timing and instalment mandates are checked whenever the sandbox clock moves; on-chain anchoring runs in a background thread and can be retried; judge accounts older than 24 hours are purged whenever a new judge sandbox starts.

Why the standard library: nothing to install, a tiny attack surface and no supply-chain risk; every security control (PBKDF2, HMAC, TOTP, Ed25519, CSP) is visible in the repository and covered by tests. In production the same design would sit behind a managed load balancer with keys in a KMS/HSM and a database in place of JSON files.

6. Payment lifecycle

Every payment passes five checkpoints, and nothing leaves the account before the last one.

1. **Invoice checked** against the university's verified record; payee details always from the registry.
2. **Route chosen:** licensed channels ranked by landed cost; disallowed channels excluded with a reason.
3. **Currency risk priced** with EWMA volatility and 4,000 Monte Carlo paths.
4. **Guardian approves** with their own one-time code when a guardian is linked; a decline is final.
5. **Student confirms** with a one-time code or authenticator app; the policy engine runs again; the transfer is tracked to "paid".

Proposal field (signed)	Meaning
`proposal_id`, `student_id`, `invoice_id`	What is paid, for whom; binds the proposal to its owner
`university_id`, `payee_account`, `payee_bank`, `reference`	Registry payee and the student reference
`provider_id`, `quoted_rate`, `legs`,	Channel, rate and amounts (one leg, or instalments)

Proposal field (signed)	Meaning
`total_dst`, `total_src_quote`	
`strategy`, `timing` (target, floor, deadline, chance)	Pay now, instalments or smart timing
`max_slippage`, `invoice_verdict`, `second_factor`, `service_fee`, `ai_hold`	Guards and disclosures carried into the signature

The proposal is serialised and signed with HMAC-SHA256 (`security/proposals.py`); it expires after 10 minutes (`TS_PROPOSAL_TTL`), executes once, and locks after 3 wrong codes.

7. AI layer

Gemini participates in four roles, each fenced by code; without a key every feature still works through the rule engine (`agent/intents.py`).

Role	What it does	What fences it
Planner	Plan → act → observe → re-plan in English, Bangla, Chinese or Malay; sees the student's last three requests.	Fixed tool list with no execute tool; ≤ 3 rounds, ≤ 5 tools; one payment tool, always last; unknown tools dropped; every call passes the permission gate, policy engine and audit log.
Quarantined reader	Reads untrusted e-mails inside random fences; returns typed JSON (sender, amount, bank details, warning flags).	No tools, no memory, output never reaches the planner; monotonic merge (can only add risk).
Independent auditor	Reviews every prepared payment and every autonomous payment.	Only a literal `"hold": true` is read; hold signed into the proposal; human acknowledgement required.
Writer	Rewrites verified replies, money briefs and e-mail drafts in the user's language — the script the student wrote in.	Number and ID preservation check; forbidden phrases (codes, passwords) rejected; template shown otherwise.

- **Model selection and resilience:** preferred model from `TS_LLM_MODEL`, then `gemini-3.1-flash-lite`, `gemini-3-flash-preview`, `gemini-flash-latest`; one retry after 2 s on HTTP 429/503; fall back to the next model on 400/404/429/5xx, network errors or an empty answer; thinking models run with low thinking and a larger output budget; 401/403 give a clear key error.
- **Prompt safety:** untrusted text never enters the planner; secrets and full account numbers are redacted before any model call; JSON mode with strict validation for every structured role.
- **Assistants:** the student AI uses the same tier-gated tools as the agent; the guardian AI is read-only and answers only from that student's facts; the public AI answers from a fixed fact sheet and is rate-limited.
- **Evaluation:** `scripts/eval_live_llm.py` runs a preflight call, measures planner accuracy (AI on vs off), follow-ups, the reader on all 84 hold-out invoices and jailbreaks, marks every result as answered by Gemini or by rules, and never overwrites evidence when the model is unavailable.

8. Engines and algorithms

8.1 Scam shield (engines/invoice_check.py)

Each warning sign adds points; the verdict is **SAFE** below 25, **REVIEW** from 25 to 59, **BLOCK** at 60 or more. Every point is explained in plain language.

Signal	Points	Signal	Points
Bank account ≠ registry account	45	Asks for crypto, gift cards or a personal account	30
Unknown payee	40	Hidden instructions aimed at AI assistants	30
Look-alike domain (incl. homoglyphs)	35	Amount anomaly / currency mismatch	15 / 15
Unofficial domain	25	Urgency / threats	10 / 10
AI reader: new bank details / unusual method / AI instructions	30 each	Hidden content / reference mismatch	10 / 10

Domains are compared by a Unicode skeleton and Levenshtein distance; zero-width and bidi-override characters count as hidden content; cues are detected in English, Malay, Bangla and Chinese, with a negation-aware rule so a genuine “we never accept crypto” warning is not flagged.

8.2 Route pricing (engines/routes.py)

Every channel is ranked by true landed cost in the home currency:

`landed cost = amount / (mid × (1 − FX markup)) + fixed fee + P(late) × (late fee + LATE_HARM_USD)`, with `LATE_HARM_USD = 400`.

`P(late)` combines the delivery window against the days left (minus a 2-day buffer) with the channel’s reliability. A channel is ineligible when it is unlicensed, not allowed by the home-country rule pack, over its limit, or not a partner of the university.

Channel (sandbox)	Type	Licensed
Bank wire (SWIFT via authorised bank)	bank	Yes
EduPay (education payment platform)	edu_platform	Yes
RemitLite (licensed money transfer operator)	licensed_mto	Yes
International debit card	card	Yes
Informal channel (hundi / hawala)	informal	No — always excluded

Home country rule pack	Allowed channel types for tuition
Bangladesh (BD)	bank, edu_platform
Nigeria (NG)	bank, edu_platform, licensed_mto
India (IN)	bank, edu_platform, licensed_mto
Pakistan (PK)	bank, edu_platform

8.3 Currency risk (engines/fx_risk.py)

- Volatility: EWMA of daily log returns (RiskMetrics, λ = 0.94), annualised with √365.
- Drift: historical mean log return shrunk 50% towards zero.
- Simulation: 4,000 geometric-Brownian-motion paths to the payment horizon (fixed seed for reproducibility).
- Output per strategy (now, later, instalments): mean cost, 95th-percentile cost (budget-at-risk) and the probability of exceeding the family's tuition budget.

8.4 Smart timing (engines/fx_timing.py)

`improvement = clamp(0.5 × σ × √(h / 365), 0.25%, 2%)`; `target = quoted × (1 + improvement)`; `floor = quoted / (1 + 5%)`. 3,000 Monte Carlo paths give the chance the target is reached before the deadline, the expected saving against paying today, and the worst case.

8.5 Planning engines

- **Semester bills** (`obligations.py`): tuition plus typical living bills, sorted by due date and checked against wallets and scheduled family support, converted at mid rate less a 1.2% licensed-channel markup.
- **Budget and runway** (`budget.py`): spending by category in rolling 30-day periods, spikes, months of runway after tuition.
- **Credit readiness** (`credit.py`): a transparent 0–100 heuristic with country playbooks (UK, US and others) — educational, not an official credit score.

9. Family and accounts

- **Linking:** at sign-up a student names a guardian and e-mail; a guardian who registers with that e-mail is linked automatically. Alternatively a single-use 8-character link code (24-hour expiry, single use, guessing locked out).
- **Co-approval:** when a guardian is linked, every proposal needs the guardian's own code first; the student's code is only usable after that approval; a decline is final; a new quote supersedes older pending ones.
- **Family funding and oversight:** guardians see invoices, balances and payments (phone numbers masked), send family support, and can freeze everything with a kill switch that only a guardian can release.
- **Second factor:** codes arrive on an in-app sandbox phone (optional real SMS via Twilio, 5 per number per hour, never logged); students can link an authenticator app (RFC 6238), after which SMS codes stop working.
- **Accounts:** PBKDF2-SHA256 (310,000 iterations, 16-byte salt); 8+ characters with upper, lower and a digit; 5 wrong passwords lock the e-mail for 15 minutes, 20 lock the IP; sessions rotate on login and are revoked on password change.

10. Certificates and on-chain proof

After a payment settles the student can issue a certificate `SR1.<base64url(claims)>.<base64url(Ed25519 signature)>` (RFC 8032 implemented on the standard library and checked against the official test vectors). Types: tuition payment, payment history, proof of funds (proves "at least" an amount; the real balance is never written; valid 30 days; SudoRoute refuses a claim the wallets cannot back).

Each certificate carries an `audit_anchor` (sequence and hash of the audit-log head). An anchor record `{cert_sha256, audit_head, audit_entries, issuer_kid, created}` is hashed and timestamped on Bitcoin through OpenTimestamps calendars; the `.ots` proof and record are downloadable from `/verify`. No wallet, coins or personal data go on-chain. The test record is confirmed in **Bitcoin block 972087**. Certificates can be downloaded as a one-page PDF.

11. Failure handling and resilience

Situation	Behaviour
Receiving bank returns a transfer (sandbox code AC01)	Money returns to the wallet; the agent explains why and refuses a blind resend
Intermediary bank deducts a fee (sandbox USD 35)	Receipt reconciled against the invoice; only the shortfall is offered; repeating the full amount is blocked by the overpayment check
Same request sent twice	Idempotent execution — one proposal executes once
Live FX feed broken or spoofed	Rates outside 0.5×–2× of the reference are rejected per currency; on any error the dated reference snapshot is used and labelled
AI model unavailable, slow or wrong	Retry and model fallback; then the rule engine; replies that change a number are discarded
No network for on-chain anchoring	Record kept as “not submitted” with a retry; nothing in the payment flow depends on it
Server restart	Users, audit log and anchors persist in `data/`; judge sandboxes are rebuilt from their accounts

12. Judge mode and isolation

`POST /api/public/judge` creates a judge student (Md Soron) and his mother (Mst Nishi) in a private **World** — its own sandbox bank, FX market, clock and agent (`server.py: World`, `WorldView`). Moving the clock or resetting affects only that world; the audit chain, certificate issuer and anchors are shared. Limits: 8 judge sandboxes per network per hour; judge accounts and worlds are purged after 24 hours; `/api/judge/switch` switches roles for judge accounts only. Only the home-page button signs in by itself; every other “Get started” opens a sign-in / create-account window.

13. Data model and persistence

Entity	Key fields	Stored in
User	id, role (student/guardian), name, e-mail, password hash, phone (masked to others), world, TOTP secret	`data/users.json`
Student (sandbox)	id, university, home/study currency, balances, guardians, student reference	Rebuilt in memory from users
Invoice	id, university, amount, currency, due date, sender e-mail, bank details on the invoice, text, verdict	Sandbox bank (memory)
Proposal	Signed payload (section 6), status, guardian approval	Proposal store (memory)
Transfer	id, provider, amounts, created, arrival, status (in transit, credited, returned), deductions	Sandbox bank (memory)
Mandate	Instalment or smart-timing plan, limits, status	Agent (memory)

Entity	Key fields	Stored in
Audit entry	seq, time, actor, event, PII-redacted details, previous hash, hash	`data/audit_log.jsonl`
Anchor	Record, digest, calendars, `.ots` proof, status	`data/anchors.json`
Issuer key	Ed25519 private key (0600 permissions)	`data/issuer_ed25519.key`
Contact, newsletter, pilot feedback	Form submissions and ratings	`data/*.json`

14. API

All `/api` routes except sign-in, registration and public endpoints need a session cookie; every POST also needs the `X-CSRF-Token` header and a same-origin request.

Method · path	Who
POST `/api/auth/register`, `/api/auth/login`	Public
POST `/api/auth/logout`, `/api/auth/password` · GET `/api/auth/me`	Signed in
GET `/api/state`, `/api/insights`, `/api/audit`, `/api/certificates`	Student
POST `/api/chat`, `/api/confirm`, `/api/settings`, `/api/mandate/cancel`, `/api/killswitch`, `/api/certificates/issue`, `/api/transfer/recover`, `/api/brief`, `/api/email-draft`, `/api/guardian/invite`, `/api/guardian/unlink`	Student
POST `/api/totp/setup`, `/api/totp/enable`, `/api/totp/disable`	Student (password / code)
GET `/api/guardian/overview` · POST `/api/guardian/decide`, `/api/guardian/link`, `/api/guardian/fund`, `/api/guardian/killswitch`, `/api/guardian/ask`	Guardian
POST `/api/public/judge` · `/api/judge/switch`	Public (8/hour) · judge
GET `/api/health`, `/api/public/meta`, `/api/public/fx`, `/api/public/issuer`, `/api/public/benchmark`, `/api/public/baselines`, `/api/public/pilot`, `/api/public/anchor`	Public
POST `/api/public/verify`, `/api/public/security-lab/run`, `/api/public/ask`, `/api/public/contact`, `/api/public/newsletter`	Public (rate-limited)
POST `/api/sandbox/advance`, `/api/sandbox/reset`, `/api/sandbox/transfer-issue`, `/api/university/simulate-verification`, `/api/chain/retry`	Signed in, sandbox only (`TS_DEMO_MODE=0` disables the clock tools)

15. Security design

The design principle is **AI proposes, code disposes, humans execute**: the AI can plan and prepare, code decides what is allowed, and only people can release money. The full control list and the 18-item known-risk list are in the separate **Security Self-Assessment**.

15.1 Permission tiers

Tier	What it allows	Who may use it
T0 Read	View balances, invoices, history	Agent, always
T1 Analyze	Verify invoices, compare routes, FX risk, budget	Agent in Advisor or Assisted mode
T2 Prepare	Draft an HMAC-signed payment proposal; no money moves	Agent in Assisted mode only
T3 Execute	Move money	People only: guardian's code (when family money is used), then the student's code or authenticator app
T4 Prohibited	Unregistered payees, bank details from documents, unlicensed channels, crypto, raising limits or level, editing the audit log, following instructions inside documents	Nobody — refused and logged

The student chooses the agent's level (Read only / Advisor / Assisted); the agent cannot raise it. The permission gate (`security/permissions.py`) checks every tool call against the tier and the level.

15.2 Defence in depth on the money path

Layer	Control	Stops
AI	Fixed tool list with no execute tool; ≤ 3 rounds, ≤ 5 tools; untrusted text only to the quarantined reader; auditor can only hold	Prompt injection, a hostile or confused model
Payee	Bank details only from the university registry; invoice compared with the registry	Changed-bank-details scams
Policy	10 checks before and again at execution (payee, invoice, overpayment, licensed and permitted channel, sanctions, limits, funds, AML pattern, documents)	Illegal channels, overpayment, structuring, limit abuse
Proposal	HMAC-SHA256 signature over every field, owner binding, 10-minute expiry, single use, 3-attempt lockout	Tampering, replay, stale quotes, code guessing
People	Guardian code first (family money), then the student's code; kill switch for both	Account takeover of one person, unwanted payments
Record	SHA-256 hash-chained audit log; Ed25519 certificates; Bitcoin timestamp	Silent edits, forged receipts
Platform	PBKDF2 passwords, lockout, sessions, CSRF, origin check, CSP, rate limits, role isolation (section 5)	Web attacks, brute force, IDOR

Evidence: Security Lab 63/63 — 57 live attacks blocked (including a deliberately hostile model in place of Gemini) and 6 controls that must still succeed; against the real Gemini model, 0 money moved and 0 leaks.

16. Web client

- **Pages:** home, sign-in / sign-up, student portal, guardian portal, Security Lab, accuracy and baselines, certificate verification, pricing, help centre, contact, legal, offline page.

- **Languages:** interface in English, 简体中文 and বাংলা; the agent understands and answers in English, Bangla, Chinese and Malay.
- **Design:** light and dark themes, responsive to phone width, scroll and state-driven motion that switches off with `prefers-reduced-motion`; an animated AI assistant on every page.
- **Platform:** installable (web app manifest + service worker that never caches money screens); no JavaScript framework or build step; strict CSP (`script-src 'self'`); content-hashed static files with long caching; gzip.

17. Testing and evaluation

Layer	What is tested	Result
Unit and integration tests (`tests/`, 13 suites)	Engines, permission tiers, signatures, replay, expiry, OTP lockout, policy, audit chain, accounts, sessions, CSRF, IDOR, guardian co-approval, planner, reader, auditor, smart timing, TOTP and Ed25519 test vectors, judge isolation, web pages, recovery	230 passed
Security Lab (`security/redteam.py`)	57 live HTTP attacks + 6 controls, including a hostile model in place of Gemini	63/63
Scam benchmark	Two hold-out sets (60 + 24 invoices) in 4 languages	100% scams flagged, 0 genuine blocked
Baselines	84 invoices, 160 priced payments, 3 retries on the real agent	0 scams paid; 1.82% cheaper; 0 rule breaks
End-to-end demo (`scripts/run_demo.py`)	9 scenarios, 41 checked steps	41/41, audit chain valid
Live Gemini (`scripts/eval_live_llm.py`)	Planner, AI on vs off, follow-ups, reader, jailbreaks	Planner 23/24 · 0 money moved · reader monotonic 100% · 0 leaks

Tests use a simulated Gemini (and the lab a deliberately hostile one), so they run offline and never call a real model even when a key is present. **Live Gemini run (20 Oct 2026, gemini-3.1-flash-lite):** 24 requests in English, Bangla, Chinese and Malay — 23 planned correctly; the one miss ("帮我付学费", pay my tuition, in Chinese) only verified the invoice and did not prepare a payment, so it failed safe. Gemini answered 115 of 135 calls; the rest fell back to the rule engine and are labelled as such in `evidence/live_llm_eval.md`. GitHub Actions runs the suite on every push (`.github/workflows/tests.yml`).

18. Deployment and configuration

1. Run locally: `python run.py` → `http://127.0.0.1:8000`.
2. Optional AI: `GEMINI_API_KEY` in `.env` (git-ignored); check with `python scripts/check_gemini.py`.
3. Publish: Render Blueprint (`render.yaml` builds the Dockerfile, generates `TS_SECRET`, sets secure cookies); the key lives in the Render dashboard, never in the repository.

Variable	Default	Purpose
`GEMINI_API_KEY`	—	Enables Gemini
`TS_LLM_MODEL`	auto	Preferred model, with fallbacks
`TS_SECRET`	random per start	Proposal-signing key (production: KMS/HSM)
`TS_LIMIT_PER_TXN_USD` / `TS_LIMIT_DAILY_USD`	30,000 / 40,000	Transaction limits
`TS_PROPOSAL_TTL`	600 s	Quote validity
`TS_MANDATE_MAX_SLIPPAGE`	0.05	Instalment pause and smart-timing floor
`TS_SERVICE_FEE_RATE` / `_MIN_USD` / `_CAP_USD` / `_WAIVED`	0.5% / 3 / 25 / waived	Visible service fee
`TS_DEMO_MODE`	1	Sandbox clock tools (0 disables)
`SMS_PROVIDER=twilio` + credentials	off	Optional real SMS

19. Limitations and roadmap

- **Sandbox only:** universities (4 sample institutions), banks, accounts and channels are simulated; FX levels are real, their future movement simulated. In production licensed partners execute payments.
- **Synthetic test sets** written by the team; set v2 still sends 2 of 12 genuine invoices to review.
- **Model variability:** safety is enforced by code, not by the model.
- **Smart timing** lowers cost on average, not on every path; the worst case is bounded by the signed floor.
- **Country rules** (BD, NG, IN, PK) are simplified illustrations to be validated with regulators.
- **Shared sandbox clock** for ordinary accounts on one server; judges use private worlds.

Roadmap: pilot with a licensed education-payment partner → real university registries and out-of-band bank verification → passkeys → more corridors → certificates recognised by lenders and credit bureaus → university reconciliation plan.

20. Sources

- Global international student numbers triple over two decades — Times Higher Education, 24 Jun 2025 (UNESCO data)
- Brain drain: outward student capital flows triple — Dhaka Tribune, 26 Jul 2026 (Bangladesh Bank and UNESCO data)
- Remittance Prices Worldwide, Q1 2025 — World Bank
- Warning: fraudulent payment scams — University of Technology Sydney
- Scams and fraud — Anglia Ruskin University
- UK university fee used as cover for hawala-hundi transfer — Associated Press of Pakistan, 2 Sep 2026
- Project evidence: `evidence/` in the repository